Hands-On Machine Learning

Chapter 1 — The Machine Learning Landscape

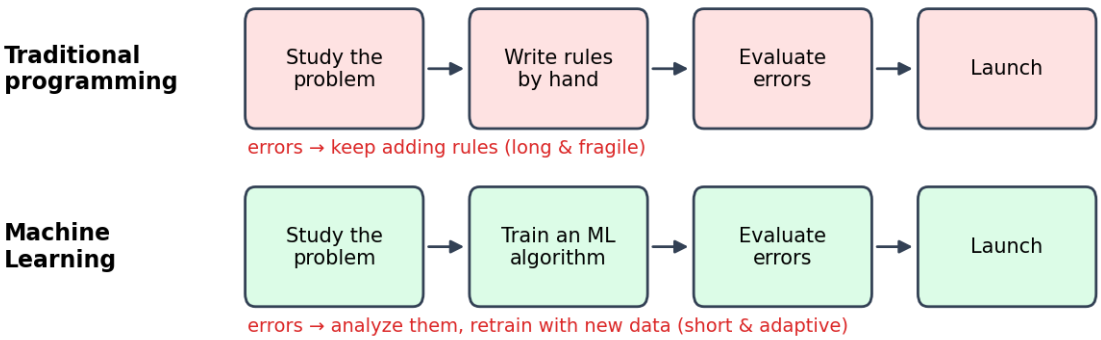
Types · Challenges · Testing & Validating

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

The numbers in the green **Output (real run)** boxes came from actually running the code (the book's real `lifesat.csv` data, and the real `digits` data from sklearn). Figures marked (synthetic) use generated data for illustration only.

1 What is Machine Learning?



The difference between traditional programming and ML: in traditional programming you write the rules by hand and keep adding rules whenever you find errors; in ML you train an algorithm on data and retrain it on new data when you find errors.

From your notebook

The person who gave the definition and was the first to coin the term is **Arthur Samuel** in 1959, and he was working in artificial intelligence.

> *Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.*

Machine learning: the field of study that gives computers the ability to learn without being programmed explicitly.

Addition

A second, more engineering-oriented definition mentioned by the book (Tom Mitchell, 1997):

> *A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.*

- `T` = the Task, e.g. classifying an email as spam / ham.
- `E` = the Experience = the data it trains on; each training example is called a training instance or sample, and all of them together the training set.
- `P` = the Performance measure, e.g. accuracy.

Example: the spam filter: T is classifying emails, E is emails that were classified before, and P is the percentage of emails classified correctly.

What is ML great at?

From your notebook

Applying ML techniques to dig into large amounts of data can help discover patterns that were not immediately apparent. This is called **data mining**.

1. **Problems whose current solutions need a lot of hand-tuning or long lists of rules:** ML often simplifies the code.
2. **Complex problems** that have no good solution at all using the traditional approach.
3. **Fluctuating environments:** a machine learning system can adapt to new data.
4. **Getting insights** about complex problems and large amounts of data.

Addition

Examples for each point (from the same context of the book): (1) the spam filter: instead of long rules that change every day. (2) Speech recognition: so complex that manual rules do not work. (3) A spam filter learns the new words that spammers invent automatically, without you modifying the code. (4) The data mining mentioned above.

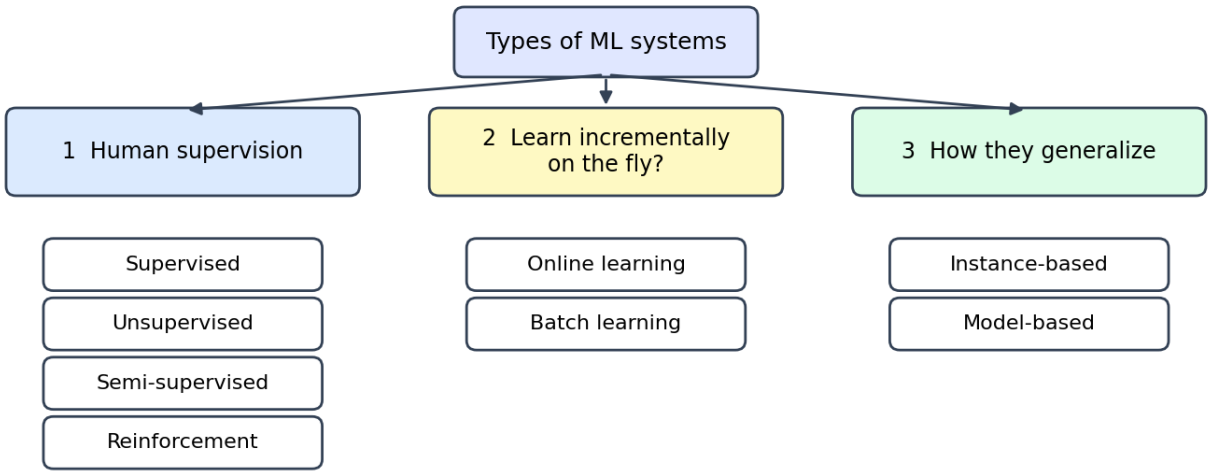
2 Types of ML systems (classified by 3 criteria)

From your notebook

1 · Human supervision during training: Supervised · Unsupervised · Semi-supervised · Reinforcement Learning.

2 · Learn incrementally on the fly? Does the system learn gradually or not? Online versus batch learning.

3 · How they generalize: Comparing new data points with known data points, or instead detecting patterns in the training data and building a predictive model, much as scientists do: instance-based versus model-based learning.



The 3 criteria are independent of each other: one system can be supervised + online + model-based at the same time (like a spam filter).

From your notebook

Remark: these are types/styles of ML learning, not exclusive types; one system can take something from each criterion.

3 Supervised Learning

From your notebook

> *Supervised learning: the training data you feed to the algorithm includes the desired solutions, called labels.*

The training data you feed to the algorithm includes the required solutions, which are called labels.

This means I need to give the model data together with its known answers. Example: I give it pictures of the thing it is going to learn, so I have: `X -> y`, where X is the input, and the output is called the label (output).

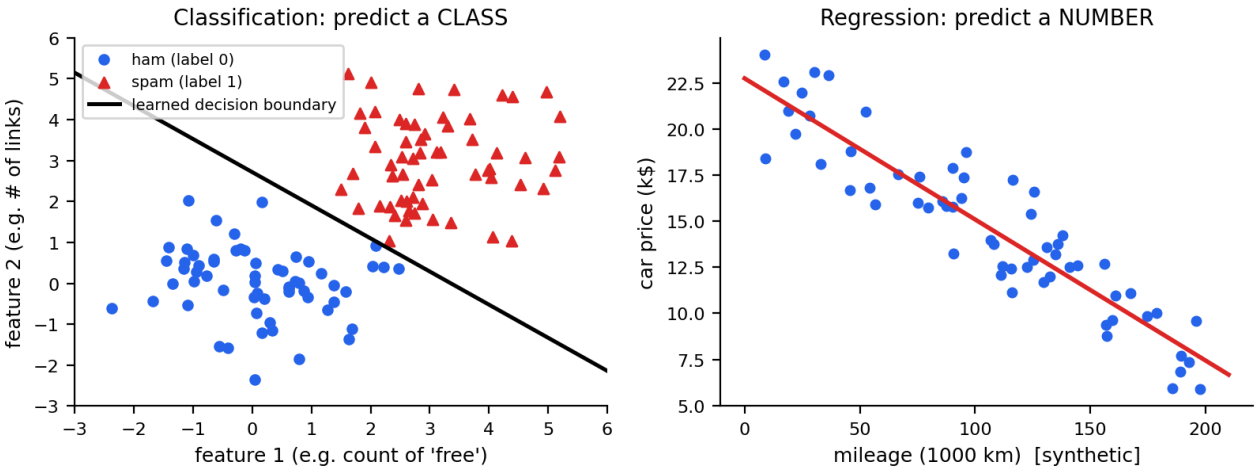
From your notebook

① Classification

We give the model data containing examples of the thing and it learns from them: if it finds this then that comes out, and if it reaches 1+1 then it means 2, and so on. Another example is the spam filter: we give the model data containing emails, each one classified, and if it finds a shape like this it is spam and this one is not spam, and it learns how to classify a new email.

② Regression

We give the model data containing data of various kinds, and it makes a **prediction** of a numeric value (target), e.g. you tell it: what is the price of this car? It builds on the data set, meaning it depends on the type of car and compares it with the data (all the price data) and compares everything and tells you the price in the end.



Left: Classification predicts a class (spam / ham) and the black line is the decision boundary the model learned. Right: Regression predicts a number (the car price). Both are real runs on synthetic data.

From your notebook

The most important Supervised algorithms (in the book):

1. k-Nearest Neighbors
2. Linear Regression
3. Logistic Regression
4. Support Vector Machines (SVMs)
5. Decision Trees and Random Forests
6. Neural networks

Addition

- `target` and `label` are often synonyms, but `target` is more common in regression and `label` is more common in classification. And `features` are also called predictors or attributes.
- Some regression algorithms can be used for classification and vice versa: Logistic Regression for example gives you the probability that an email is spam (an estimate between 0 and 1) and then you convert it into a class. (We will see it in detail in Chapter 4.)
- Some Neural Network architectures can be unsupervised (like autoencoders and restricted Boltzmann machines) or semi-supervised.

4 Unsupervised Learning

From your notebook

> *Unsupervised learning: the training data is unlabeled. The system tries to learn without a teacher.*

The training data is unlabeled; the system tries to learn without a teacher. That is, I have data but there are no labels (output) for each data point. For example, you now have a lot of customers and visitors, and you have data about them (including gender, age, region...), and what is required is an algorithm that tries to discover groups of similar visitors. And you do not tell the algorithm which visitor belongs to which group: it finds the relationships by itself, without your help.

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
kmeans.fit(X)                       # NO labels y are passed!
print(kmeans.cluster_centers_)      # the 3 group centers it discovered
```

Output (real run)

```
[[-8.54  5.57]
 [-1.26  4.39]
 [ 9.62  0.56]]
```

From your notebook

The most important Unsupervised algorithms:

① **Clustering:** K-Means · Hierarchical Cluster Analysis (HCA) · Expectation Maximization.

② **Visualization and dimensionality reduction:** Principal Component Analysis (PCA) · Kernel PCA · Locally-Linear Embedding (LLE) · t-distributed Stochastic Neighbor Embedding (t-SNE).

③ **Association rule learning:** Apriori · Eclat.

Note / Correction

The list in your notebook (it includes Expectation Maximization) is the list from the older editions. In the third edition of the book, the examples mentioned for clustering are k-means, DBSCAN and HCA. EM is the algorithm used to train Gaussian Mixture Models (Chapter 9), but add DBSCAN to your list. The book also counts Anomaly detection as an independent unsupervised task (isolation forest, one-class SVM), and that is in your notebook below.

From your notebook

Clustering

For example if you have many pictures and want the clustering to divide these pictures into groups of similar things, without telling it what exactly these pictures are. (A book example: dividing visitors.)

From your notebook

Visualization algorithms

> *Visualization algorithms are also a good example of unsupervised learning algorithms: you feed them a lot of complex and unlabeled data, and they output a 2D or 3D representation of your data that can easily be plotted. These algorithms try to preserve as much structure as they can (e.g., trying to keep separate clusters in the input space from overlapping in the visualization), so you can understand how the data is organized and perhaps identify unsuspected patterns.*

They convert complex data with many dimensions (for example, in the explanation (not the book): 100 or 200 or 300 features) into 2D or 3D so you can draw them and see the shape of the data with your own eyes, to see: is the data gathered into groups? How many clusters exist?

And it also lets you understand the data: overlapping (is there two groups of data inside each other and their boundaries are not clear), and is it linear or nonlinear (curves). So if there is overlapping the classifier will mix up a lot and you will not be able to draw a clear separating line between them, so the model needs more complex Features or needs extra preprocessing.

From your notebook

Dimensionality Reduction

It is the process of reducing the number of features while keeping as much of the important information as possible (for example, the number of features was 200 or 300 and becomes 50 or 20). Why? Because many features cause a lot of computation and slow training, and they take up a lot of space, and increase the risk of overfitting because the model gets lost among many features and learns the noise.

Addition

The feature extraction example from the book: a car's mileage is very correlated with its age, so the dimensionality reduction algorithm merges them into one feature representing "wear and tear". In the example below: PCA on the car's age and the number of kilometers (very correlated) kept 98% of the information in one axis only.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=1)                # 2 correlated features -> 1 axis
X_1d = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)
```

Output (real run)

```
[0.9759]   <- one axis keeps 97.6% of the information
```

Addition

An important hint from the book: try to reduce the dimensions of the data before you feed it to another algorithm (supervised); it will be faster, take less space, and in some cases perform better.

From your notebook

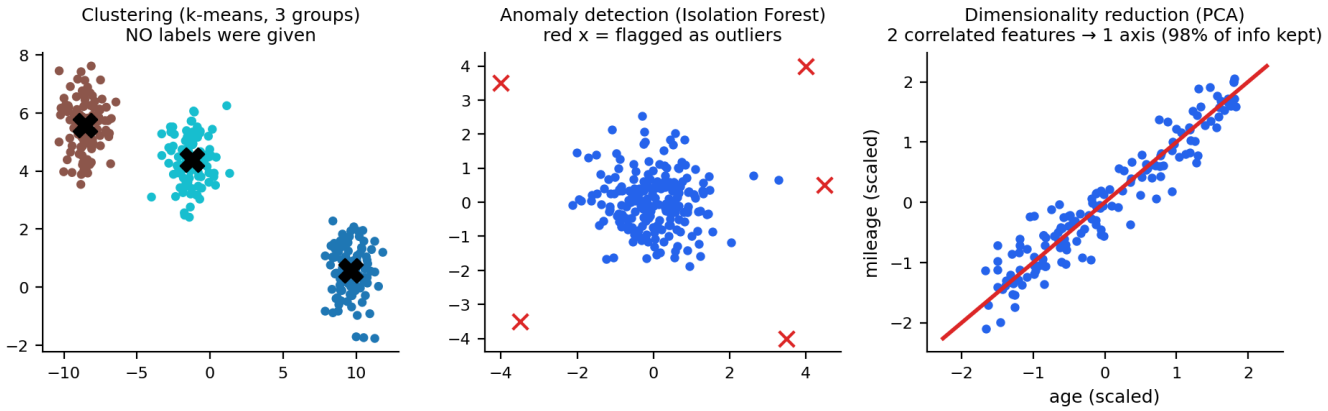
Anomaly detection

The idea is very simple: you have data that is all normal and you want to know if a strange item appears that does not look like the rest of the data. You train on a lot of normal data (and strange cases are few), so you make a system that memorizes the normal shape, and when it sees a point far from it, it says this is an Anomaly / outliers.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.025, random_state=42)
flags = iso.fit_predict(X)             # -1 = anomaly (outlier), 1 = normal
print((flags == -1).sum(), "points flagged")
```

Output (real run)

```
5 points flagged
```



The 3 basic unsupervised tasks, from a real run (synthetic data): clustering without labels, anomaly detection (red x = flagged outliers), and dimensionality reduction (PCA kept 98% of the information in one axis).

From your notebook

Association rule learning

Algorithms that look for relationships (associations) between items inside the data. **A famous example:** if someone buys Bread and Butter, there is a high probability they buy Milk, so you state the rule (Bread + Butter ⟹ Milk). It also helps, for example, in grouping the items that appear together in Clustering.

Note (an approximate reading of your words): discovering relationships and Patterns without labels is the most important step in the Machine Learning Pipeline, because without seeing the Patterns before choosing a model you will choose the wrong model and do the wrong preprocessing, and all the Accuracy will drop. (Check the page: the line in the last two lines of the Unsupervised page is not completely clear.)

Addition

Anomaly detection vs Novelty detection (from the book): anomaly detection trains on data that may contain a few outliers and does not have to be perfectly clean; but novelty detection needs a clean training set (without the strange thing you are looking for). Example: if you have millions of dog pictures of which 1% are Chihuahuas, novelty detection should not treat a new Chihuahua picture as a novelty, whereas anomaly detection may consider it an anomaly.

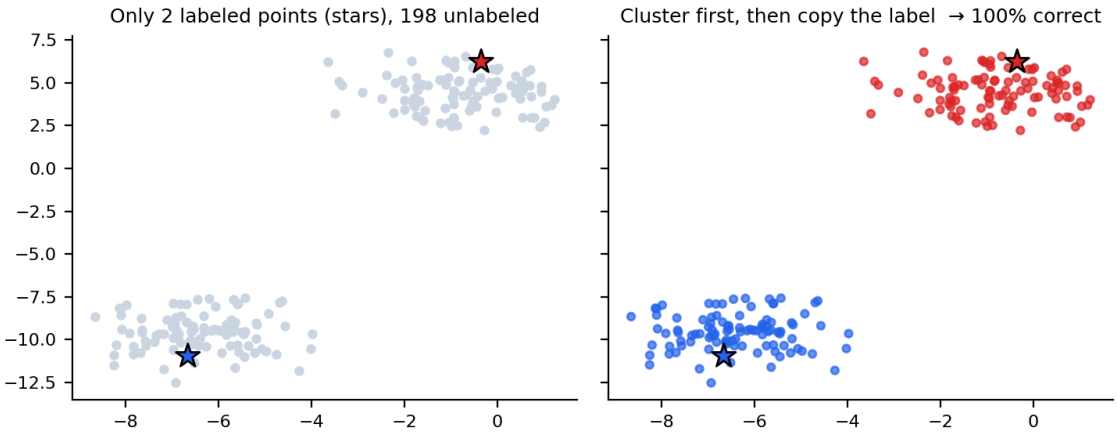
5 Semi-supervised and Self-supervised

From your notebook

> *Semi-supervised learning: some algorithms can deal with partially labeled training data, a lot of unlabeled data and a little bit of labeled data. This is called semi-supervised learning.*

Some algorithms can deal with partially labeled training data, usually a lot of unlabeled data and a little labeled data.

Most semi-supervised learning algorithms are a combination of unsupervised and supervised algorithms. For example: Deep Belief Networks (DBNs): they are based on unsupervised components called Restricted Boltzmann Machines (RBMs) stacked on top of each other. The RBMs are trained one after the other in an unsupervised way, then the whole system is fine-tuned using supervised learning techniques.



The idea of semi-supervised: only two labeled points (the stars) are enough to name every cluster if the data is clustered. (Synthetic data, easy on purpose.)

Addition

The book example: photo-hosting services like Google Photos: they recognize that the same person appears in many pictures (this is the clustering part: unsupervised), then you only need to tell it the person's name once (one label) and it puts the name on every picture (supervised). This is very useful because labeling is expensive and takes human time.

Self-supervised learning (a separate type in the third edition, not in your notebook): the algorithm generates labels from unlabeled data itself. For example: take a picture, erase a piece of it, and make the model try to guess the missing piece (the real piece is the label). After that, this model learns a good representation of images, and you can fine-tune it on a real task with a little labeled data (like classifying animal types).

6 Reinforcement Learning

From your notebook

> *Reinforcement learning is a very different beast. The learning system, called an agent in this context, can observe the environment, select and perform actions, and get rewards in return (or penalties in the form of negative rewards).*

Reinforcement learning is something completely different. The learning system, which is called the **agent** in this context, can observe the environment, select and perform actions, and receive rewards in return (or penalties in the form of negative rewards).

After that it must learn by itself what the best strategy is, called a **policy**, to get the largest amount of rewards over time. The policy defines the action the agent should choose when it is in a given situation.

Some applications of reinforcement learning: DeepMind's AlphaGo program, which beat the world champion in the game of Go.



goal: learn the policy that maximizes the total reward over time

The RL cycle: the agent performs an action, and the environment returns observation + reward. The goal is a policy that achieves the largest cumulative reward.

7 Comparison table (from your notebook + the empty columns completed)

From your notebook

The table you made in your notebook in English (columns 1 to 4; columns 5 and 6 (supervised, unsupervised) were empty, and I completed them below in orange).

	① Online learning	② Batch learning	③ Reinforcement	④ Semi-supervised
Advantages	Handles large or infinite datasets · Good for real-time systems · Adapts quickly to new patterns · Low memory usage	Very accurate when data is fixed · Training process is stable and easier to tune · You can preprocess the whole dataset before training	Learns optimal behaviors through trial and error · Good for sequential decisions · Does not require labeled data, only rewards	Reduces labeling cost · Good for domains where labels are expensive
Disadvantages	Harder to tune · Model may become unstable with noisy data · Cannot easily go back and correct earlier mistakes	Requires large memory and computation · Cannot adapt to new data unless retrained from scratch · Not suitable for real-time learning	Very slow and computationally expensive · Designing reward functions is hard · Can behave unpredictably if rewards are wrong	Methods can be complex · Wrong labels or assumptions can degrade performance (mistakes in the few labels get transferred to all the data)
Examples	Spam filters that update daily · Online recommendation systems · Stock market prediction systems	Training a neural network on ImageNet · Training a face recognition model on a large dataset	AlphaGo, AlphaZero · Self-driving car navigation · Robot control	(left empty in the notebook)

Addition

The columns that were empty, completed in the same style:

	⑤ Unsupervised	⑥ Supervised
Advantages	No labels needed (cheap data) · Discovers hidden structure/groups you didn't know about · Useful for visualization, anomaly detection and as a preprocessing step	Clear goal and easy to measure (accuracy, RMSE...) · Usually the most accurate when enough labeled data exists · Many mature algorithms
Disadvantages	Hard to evaluate: there is no "correct answer" to compare with · Results can be hard to interpret · You must decide the number of clusters/components	Needs lots of LABELED data (expensive and slow to collect) · Learns the labelers' mistakes and biases · Can overfit the training labels
Examples	Customer segmentation (k-means) · Anomaly detection of fraud · Dimensionality reduction with PCA	Spam filter (classification) · House/car price prediction (regression) · Digit recognition

8 Batch Learning and Online Learning

From your notebook

> *Batch learning: In it, the system is incapable of learning incrementally: it must be trained using all the available data.*

In batch learning the system is unable to learn gradually; it must be trained using all the available data. This usually takes a lot of time and computing resources, so it is usually done offline (offline learning).

First the system is trained, then it is launched into production and works like this without any further learning; it just applies what it learned. This is called offline learning.

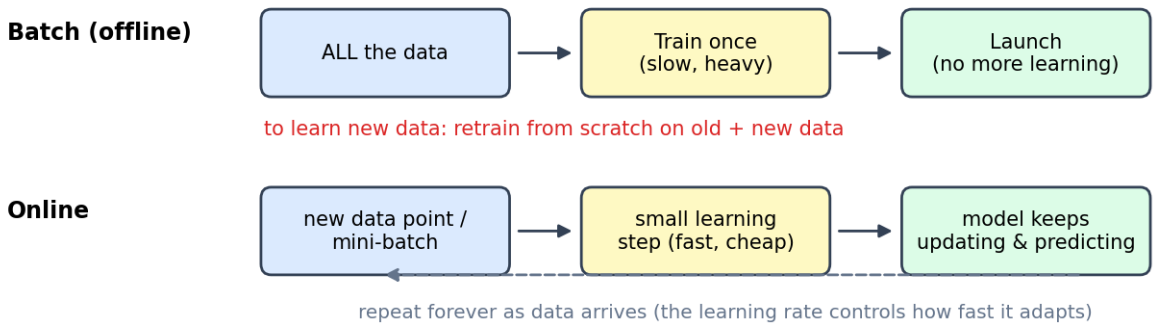
If you want a batch learning system to know about new data (such as a new type of pictures or messages), you have to retrain a new version of the system from scratch on the complete dataset (the new data and the old, then replace the old system with the new one). This is the only solution if you want it to adapt to change.

From your notebook

> *Online learning: In it, you train the system incrementally by feeding it data instances sequentially, either individually or by small groups called mini-batches. Each learning step is fast and cheap, so the system can learn about new data on the fly, as it arrives.*

In online learning you train the system gradually by feeding it data instances sequentially, either individually or in small groups called mini-batches. Each learning step is fast and cheap, so the system can learn about new data as it arrives.

If you have limited computing resources, once an online learning system has learned from the new data it does not need it anymore, so you can get rid of it (and this can save a huge amount of space).

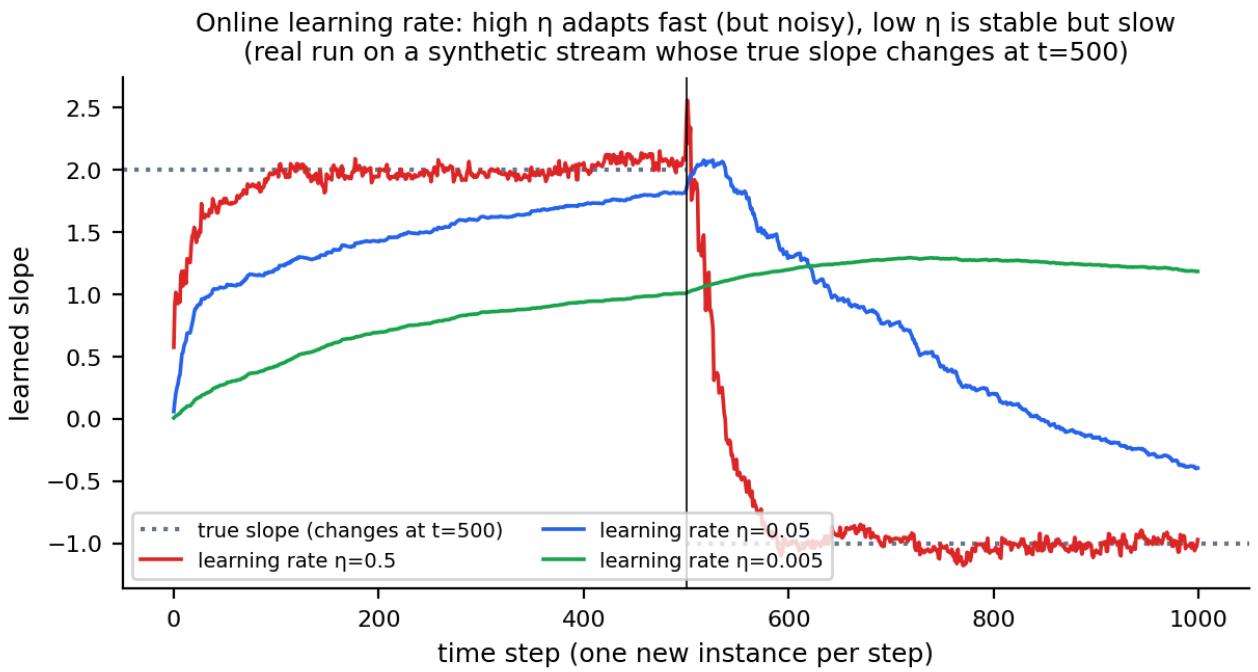


The difference in the figure: Batch trains once on all the data and then freezes, and Online keeps updating continuously with every new data point.

From your notebook

> *One important parameter of online learning systems is how fast they should adapt to changing data: this is called the learning rate.*

One of the important elements of online learning systems is how fast they adapt to changing data, and this is called the **learning rate**. If you set a high learning rate your system will adapt quickly to the new data, but it will tend to forget the old data quickly. On the contrary, if you set a low learning rate the system will learn more slowly, but it will be less sensitive to noise in the new data.



The same idea by experiment: 3 values of η on a stream whose true slope changes at t=500. Red (η large) caught up with the change quickly but has jitter, and green (η small) is stable but still trying to catch up at the end of the stream. (Real run on a synthetic stream.)

Addition

Also in the book: online learning is used with huge data that will not fit in memory at once, and this is called **out-of-core learning**: the system loads part of the data and trains on it and repeats until all the data is finished (it is also done offline, so the name "online" is a bit misleading). Another important point: if bad data enters the system, performance will drop quickly; you must monitor the system and stop learning (and you can do a rollback) if you notice a drop in performance.

9 Instance-Based vs Model-Based Learning

From your notebook

There is another way to classify machine learning systems, which is how they generalize. Most machine learning tasks are about making predictions. This means that given a number of training examples, the system needs to be able to make good predictions for examples it has never seen before. Having a good performance measure on the training data is good, but not enough; the real goal is good performance on new cases.

There are two main approaches to generalization:

From your notebook

> *Instance-based learning: the system learns the examples by heart, then generalizes to new cases using a similarity measure.*

① **Instance-based learning:** the system learns the examples by heart, then generalizes to new cases using a similarity measure.

> *Model-based learning: build a model of these examples, then use that model to make predictions.*

② **Model-based learning:** build a model of these examples, then use this model to make predictions.

From your notebook

> *The linear regression algorithm: comes in, you feed it your training examples and it finds the parameters that make the linear model fit best to your data. This is called training the model.*

Here comes the role of the linear regression algorithm: you feed it the training examples and it finds the parameters that make the linear model best fit your data. This is called training the model. (Linear regression is used when the relationship between X and y is roughly linear; for predicting a price for example, when you have a lot of data and want the model to learn from it, not compare with it like KNN.)

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor

lifesat = pd.read_csv("lifesat.csv")                # the book's data: 27 countries
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

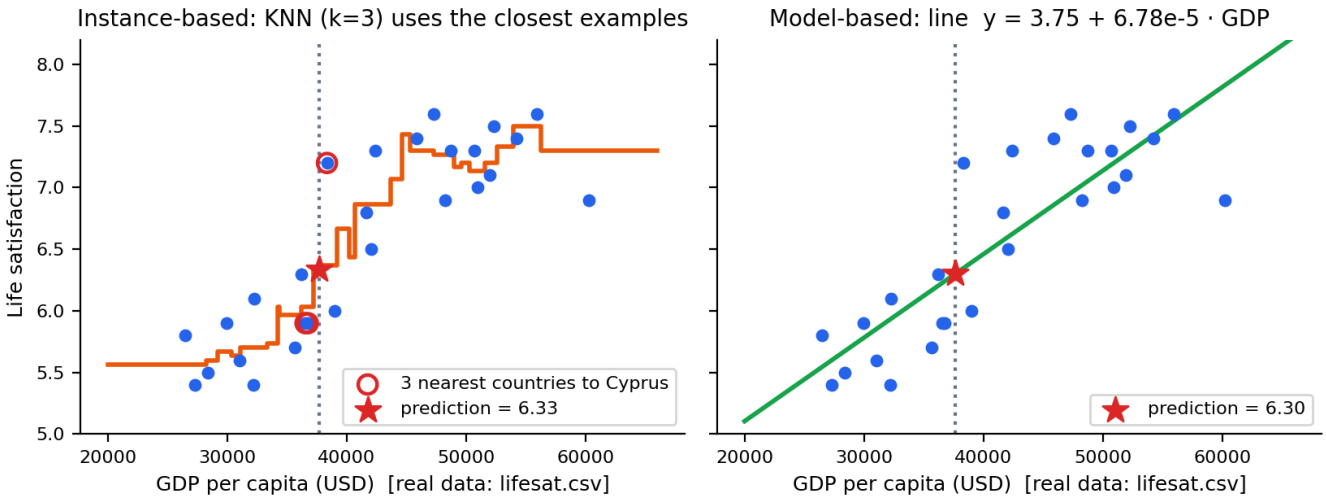
X_new = [[37_655.2]]                                # Cyprus' GDP per capita
```

```
model = LinearRegression().fit(X, y)                # model-based
print(model.intercept_, model.coef_)                # theta0, theta1
print(model.predict(X_new))

knn = KNeighborsRegressor(n_neighbors=3).fit(X, y)  # instance-based
print(knn.predict(X_new))
```

Output (real run)

```
[3.7490] [[6.7789e-05]]
[[6.3017]]
[[6.3333]]
```



The same real data in both ways. KNN (instance-based): looks at the 3 nearest countries to Cyprus in GDP (Israel, Lithuania and Slovenia) and takes the average of their life satisfaction = 6.33. Linear Regression (model-based): learns a line with parameters and draws on it = 6.30. The two results are close but the idea is different.

Addition

The difference in essence: KNN **memorizes** all the examples (and needs to carry them and compare against them at prediction time). Linear Regression **summarizes** the examples in just two numbers (θ0 and θ1) and after that you can throw away the data. And the numbers in the green box came out exactly as in the book: θ0 = 3.75 and θ1 = 6.78e-5, and the prediction 6.30.

From your notebook

Cost function

Cost Function: it measures (how accurate the relationship between the features and the labels is).

- If the value of the cost function is **high** ← the model learned a weak or incorrect relationship.
- If it is **low** ← the model learned the relationship reasonably well.

That is, it measures how accurate the Parameters the model learned during training are at representing the relationship between the labels and features.

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

From your notebook

hyperparameter tuning: the process of tuning and improving the model settings (the settings you decide before training, not the ones the model learns).

10 Introduction to Gradient Descent

From your notebook

> *Gradient descent: is a very generic optimization algorithm capable of finding optimal solutions to a wide range of problems.*

Gradient descent: a very general optimization algorithm capable of finding optimal solutions to a wide range of problems.

> *The general idea of gradient descent: is to tweak parameters iteratively in order to minimize a cost function.*

The general idea of gradient descent is to adjust the parameters repeatedly in order to minimize a cost function.

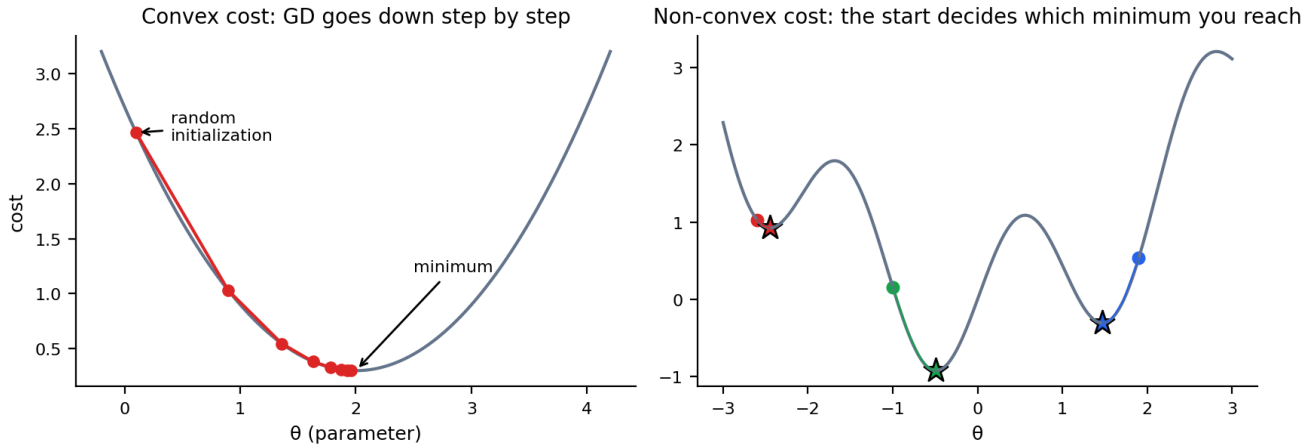
From your notebook

> *What GD does? it measures the local gradient of the error function with regards to the parameter vector θ, and it goes in the direction of descending gradient. once the gradient is zero, you have reached a minimum.*

Gradient descent: it measures the local gradient of the error function with respect to the parameter vector θ, and goes in the direction of the descending gradient. Once the gradient becomes zero, you have reached the minimum.

Concretely: you start by filling θ with random values (this is called Random initialization). Then you improve it gradually, with small steps each time, where each step tries to reduce the cost function (like MSE) until the algorithm converges to the minimum.

$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta J(\theta)$$



Left: the cost is a convex function: from any starting point it descends to the same minimum. Right: if the cost is not convex, the starting point decides which minimum you reach: red and blue got stuck in local minima, and green reached the global minimum. (Real run.)

From your notebook

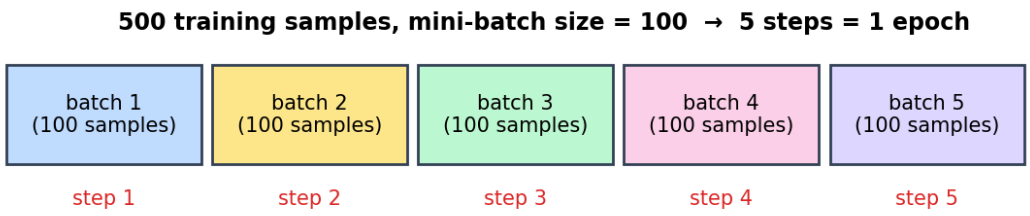
> *An important parameter in GD is the size of the steps, determined by the learning rate. If the learning rate is too small, then the algorithm will have to go through many iterations to converge, which will take along time.*

An important parameter in gradient descent is the size of the steps, which is determined by the learning rate hyperparameter. If the learning rate is too small, the algorithm will have to go through many iterations to converge, and this will take a long time.

Another remark: if the learning rate is too high, you may jump across the valley and end up on the other side, maybe even at a higher place than where you were before. This can cause the algorithm to diverge (jumping to bigger and bigger values) and fail to find a good solution.

From your notebook

A remark about the mini-batch and the epoch (from your sheet): if I have 500 data points and use a mini-batch of size 100, then the one epoch has 5 steps (because 5 × 100 = 500) because every part (every 100) gives one step.



The remark figure: each batch = one step. The epoch is one full pass over all the data.

Addition

(Check Chapter 4 for the full details: Batch and Stochastic and Mini-batch GD, the learning schedule, and scaling.) The new thing here: the cost we deal with in Linear Regression is convex, so there is no local minima problem, but in Neural Networks no, and this is what the right drawing explains.

11 The main challenges in ML

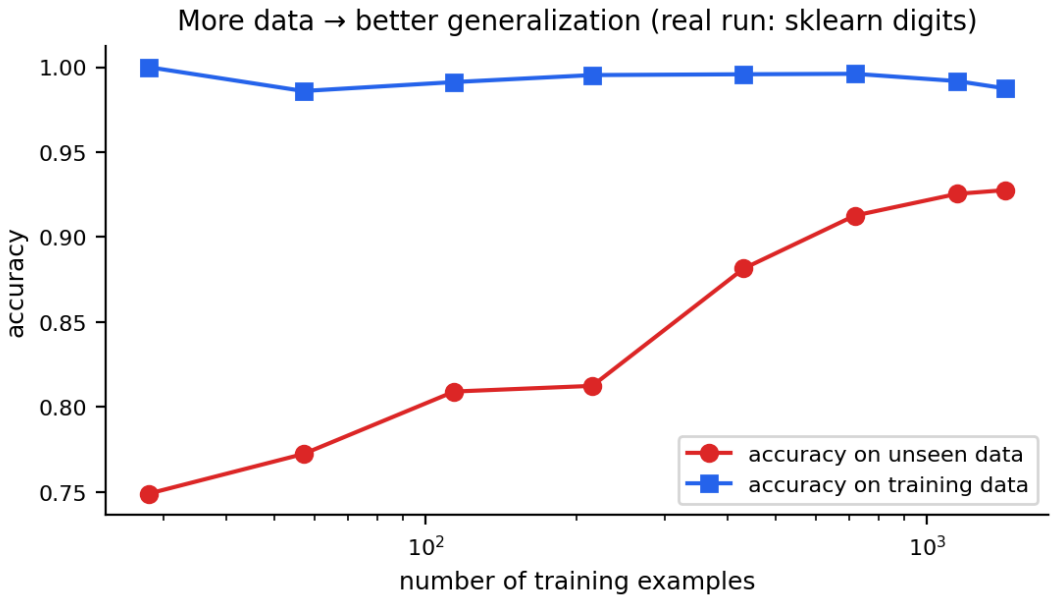
From your notebook

In short, since your main task is to select a learning algorithm and train it on some data, the two things that can go wrong are "bad algorithm" and "bad data". Let's start with examples of bad data.

From your notebook

① Insufficient quantity of training data

For a small child to learn what an apple is, all it needs is for you to point at the apple and say "apple" (and repeat it several times). But machine learning has not reached this stage yet; it takes a lot of data for most machine learning algorithms to work properly. Even for very simple problems you usually need thousands of examples.



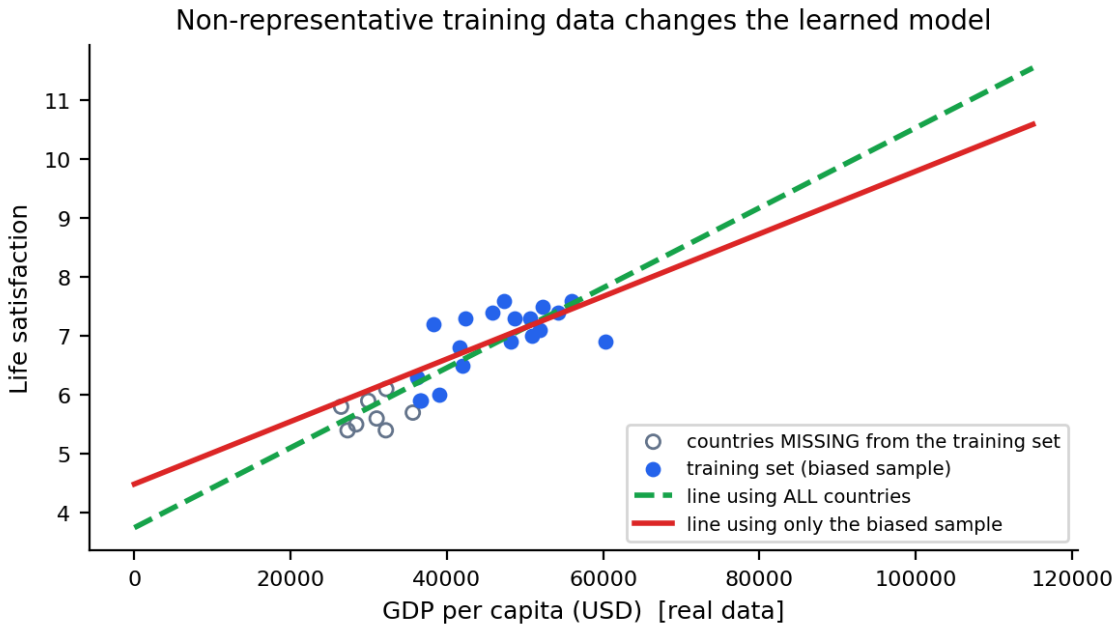
Real experiment on digits: the accuracy on data it has not seen goes from about 75% with about 28 training examples to 93% with about 1437 examples. The same algorithm exactly; the only difference is the amount of data.

From your notebook

② Nonrepresentative training data

> *In order to generalize well, it is crucial that your training data be representative of the new cases you want to generalize to. This is true whether you use instance-based learning or model-based learning.*

For good generalization, it is essential that the training data represents the new cases you want to generalize to. This is true whether you use instance-based learning or model-based learning.



Real experiment on lifesat.csv: when we removed the poorer countries from the training set, the learned slope changed from 6.78e-5 to 5.30e-5. The model is not wrong; the data it trained on is what does not represent reality.

From your notebook

③ Poor quality data

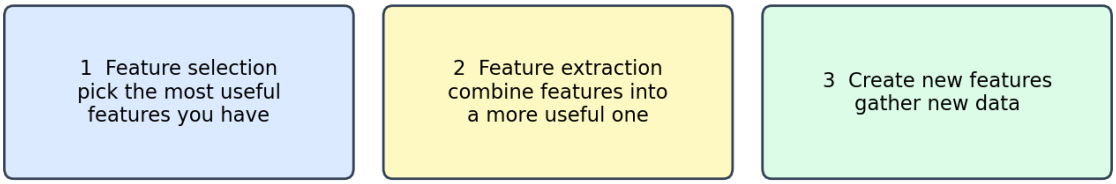
It is clear that if your training data is full of errors, outliers and noise (for example due to poor measurement quality), it will make it harder for the system to detect the underlying patterns, and so your system is less likely to perform well. It is often worth spending time cleaning the training data.

From your notebook

④ Irrelevant features

If you said "garbage in, garbage out": irrelevant data came out. Your system will only be capable of learning if the training data contains enough relevant features and not too many irrelevant features. A critical part of the success of any machine learning project is coming up with a good set of features to train on. This process includes Feature engineering and the following:

1. **Feature selection:** choosing the most useful features to train on among the existing features.
2. **Feature extraction:** combining existing features to produce a more useful one.
3. **Creating new features** by gathering new data.



The 3 ways to improve the features (Feature engineering).

From your notebook

Everything that has passed were examples of **bad data**, and now we start with some examples of **bad algorithms**.

From your notebook

⑤ Overfitting the training data

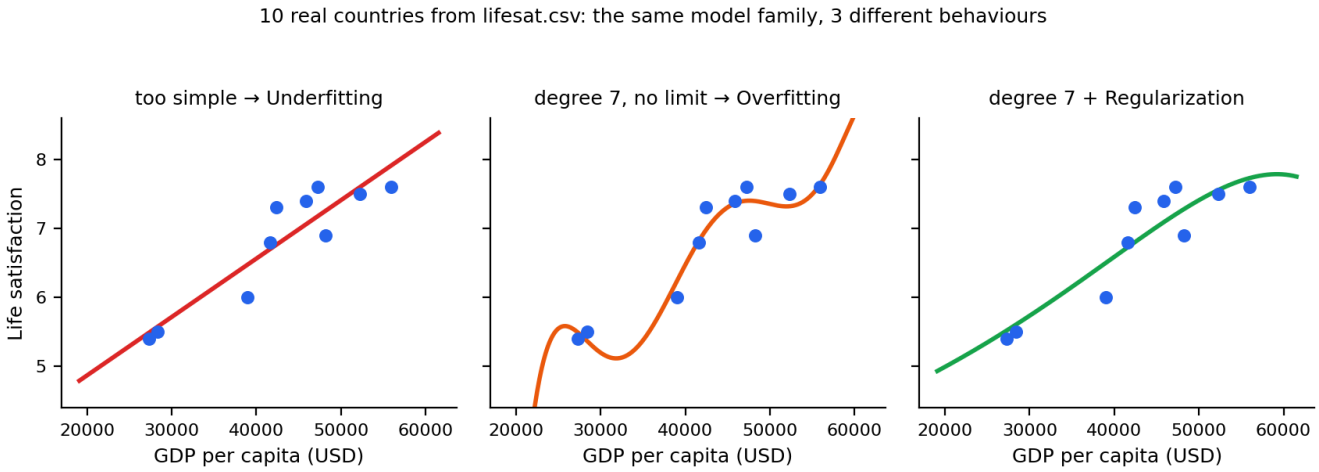
> *Overfitting the training data: it means that the model performs well on the training data, but it does not generalize well.*

Overfitting: it means the model performs well on its training data but does not generalize well. (That is, the model trained so much on the data that it memorized it, meaning if it sees the same shape it knows it, if it is different, no.)

> *Overfitting happens when the model is too complex relative to the amount and noisiness of the training data.*

Overfitting happens when the model is too complex relative to the amount and noisiness of the training data. The possible solutions are:

1. **Simplify the model** by selecting a model with fewer parameters, or by reducing the number of attributes in the training data, or by **constraining the model**.
2. **Gather more training data.**
3. **Reduce the noise in the training data** (for example: fix data errors and remove outliers).



Real experiment on 10 countries from lifesat.csv: the straight line is too simple (Underfitting), a polynomial of degree 7 with no constraints wraps around every point (Overfitting), and the same polynomial but with Regularization calms down and stays flexible.

From your notebook

Constraining a model to make it simpler and reduce the risk of overfitting is called regularization.

Constraining a model to simplify it and reduce the risk of overfitting is called **Regularization** (also used to mean simplifying).

Remark: the Parameters are the things in the model that it can tune, and all of that affects the outputs and it is what it learns; whereas a hyperparameter is something you control (you are the one who sets a value for it, like the amount of regularization and the learning rate of GD); and you must define it before training.

(* anything that you set for the model is a hyperparameter, not a parameter.)

From your notebook

⑥ Underfitting the training data

> *Underfitting the training data: it occurs when your model is too simple to learn the underlying structure of the data.*

Underfitting occurs when your model is too simple to learn the underlying structure of the data. (That is, the model is not trained enough on the data, or the data is not enough and very small.)

The main options to fix this problem are:

1. Select a more powerful model, with more parameters.
2. Feed better features to the learning algorithm (feature engineering).
3. Reduce the constraints on the model (for example: reduce the regularization hyperparameter).

	Underfitting	**Overfitting**
Model	too simple	too complex
Performance on the training set	poor	excellent
Performance on new data	poor	poor
Solutions	a stronger model / better features / weaker Regularization	a simpler model / more data / reduce the noise / stronger Regularization

Addition

Regularization is the weapon that works in both directions: if overfitting, make the regularization hyperparameter larger; if underfitting, make it smaller. We go deeper into it in Chapter 4.

12 Testing and Validating

From your notebook

Just training a model and then leaving it without testing is not enough ("hope" in its generalization to new cases). Rather you want to evaluate it and tune it carefully if necessary.

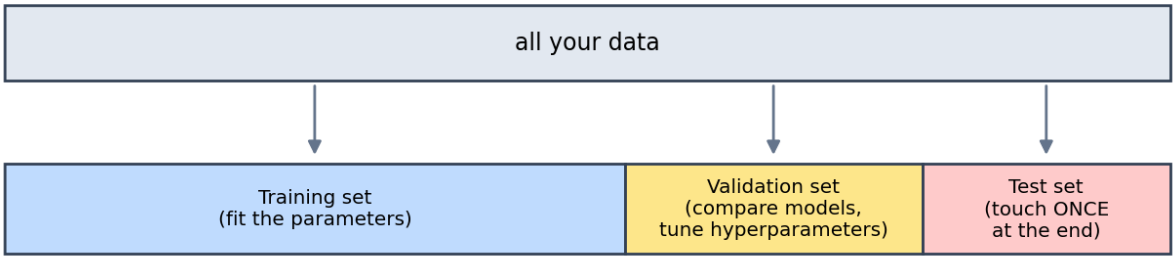
The only way to know how well a model will generalize to new cases is to actually try it on new cases. And the best way to do that is to split your data into two sets: a training set and a test set.

From your notebook

> *The error rate on new cases is called generalization error, and by evaluating your model on the test set, you get an estimation of this error. This value tells you how well your model will perform on instances it has never seen before.*

The error rate on new cases is called the **generalization error**, and by evaluating your model on the test set you get an estimate of this error. This value tells you how well your model will perform on cases it has never seen before.

If the training error is low (i.e. your model makes few mistakes on the training set) but the generalization error is high, this means your model overfits the training data.



The full split: training to train the parameters, validation to choose the model and tune the hyperparameters, and test once at the end to estimate the generalization error.

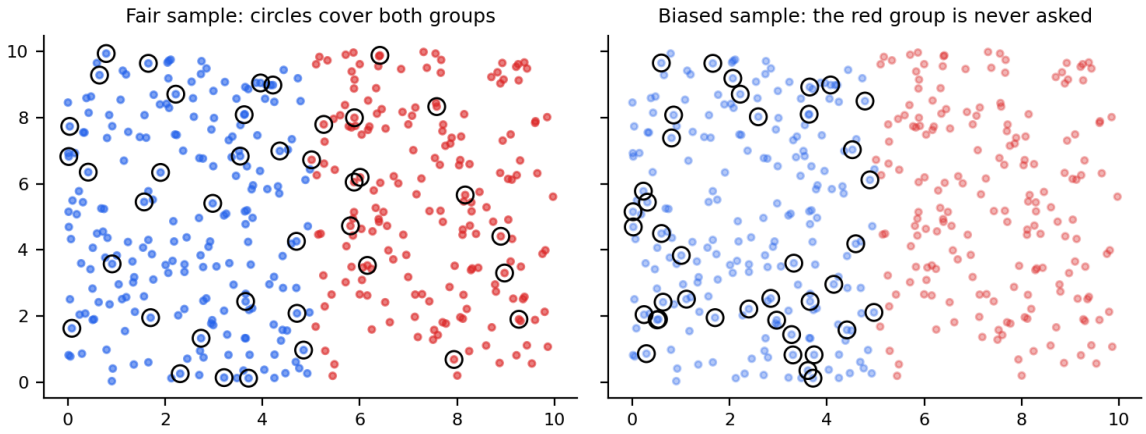
From your notebook

Cross Validation: a technique for evaluating the model by splitting the data into several parts and training and testing on them in turn, to get a more stable estimate that is closer to reality of the performance.

From your notebook

Data Mismatch: it means the data the model learned from is different from the data it will work on in reality.

Sampling Bias: bias in selecting the sample such that the sample you chose does not represent the Population fairly (the Sample was chosen in an incorrect way, so the Sample does not represent the Population ← then the results / the model will be biased).



Left: a fair sample covers both groups. Right: a biased sample (the red group was never asked), so any model that learns from it will be biased.

Addition

An important point from the book: if you tuned the hyperparameters using the test set, you will get an over-optimistic estimate of the generalization error (the model was tuned on the test itself). The solution: a fixed **validation set** (or dev set) for tuning, or cross-validation, and the test set only once at the end. And in the data mismatch case (for example training data is pictures from the internet but the product will work on mobile pictures), the book suggests a **train-dev set**: a part of the training data we set aside so we know whether the problem is from the model or from the data difference.

And a rule of thumb the book mentions: we usually split 80% / 20% (train / test), and if the data is huge (millions) we can be satisfied with 1% or less for the test.

13 Quick review questions

Answer first in your head, then check the answers on the next page.

1. Who was the first person to coin the term Machine Learning? And what is his definition?
2. Explain the (E, T, P) definition of Tom Mitchell on the spam filter example.
3. When is ML more suitable than traditional programming? (4 cases)
4. List the three criteria for classifying ML systems.
5. What is the difference between Classification and Regression? An example of each.
6. List 6 supervised algorithms from the book.
7. What is the difference between Supervised and Unsupervised?
8. List 3 main unsupervised tasks with one algorithm for each.
9. What is Dimensionality Reduction and why is it useful?
10. What is the difference between Anomaly detection and Novelty detection?
11. What is Semi-supervised? The Google Photos example.
12. What is Self-supervised?
13. Explain Reinforcement Learning: agent, environment, action, reward, policy.
14. What is the difference between Batch and Online learning?
15. What is the learning rate in online learning? What happens if it is high or low?
16. What is out-of-core learning?
17. What is the difference between Instance-based and Model-based? An example of each.
18. What is the Cost function? And what is the difference between a Parameter and a Hyperparameter?
19. Explain the idea of Gradient Descent and what happens if the learning rate is very small or very large?
20. What is an epoch? If 500 data points and mini-batch = 100, how many steps are in the epoch?
21. List 4 challenges concerning the data and 2 concerning the algorithm.
22. What is the difference between Overfitting and Underfitting? And what are the solutions for each?
23. What is Regularization?
24. Why do we split the data into train / validation / test? And what is the generalization error?
25. What is Sampling bias? And Data mismatch?

14 Review question answers

1. Arthur Samuel in 1959: "field of study that gives computers the ability to learn without being explicitly programmed".
2. A program learns from experience E with respect to a task T and a performance measure P if its performance on T, measured by P, improves with E. In the spam filter: T = classifying emails, E = emails classified before, P = the percentage of correct classification.
3. 1) Problems that need many rules/manual tuning. 2) Complex problems with no good solution by traditional methods. 3) Fluctuating environments. 4) Getting insights (data mining) from huge data.
4. 1) Human supervision (supervised, unsupervised, semi, RL). 2) Does it learn gradually (online) or all at once (batch). 3) Instance-based or model-based.
5. Classification predicts a class (ham/spam), Regression predicts a number (car price).
6. k-Nearest Neighbors, Linear Regression, Logistic Regression, SVMs, Decision Trees/Random Forests, Neural networks.
7. Supervised data has labels (the answers); Unsupervised has no labels and the system learns without a teacher.
8. Clustering (k-means), Visualization/dimensionality reduction (PCA), Association rule learning (Apriori) — and also Anomaly detection (Isolation Forest).
9. Reducing the number of features while keeping the most information; it speeds up training, reduces the risk of overfitting, and saves space.
10. Anomaly detection trains on data that may contain a few outliers; novelty detection needs a clean training set (without the strange thing).
11. Data with a few labels and most of it without labels. The Google Photos example: clustering finds the same person, then one label is enough and it generalizes.
12. The system generates labels from the unlabeled data itself (for example it erases a part of the picture and tries to guess it).
13. The agent observes the environment and performs an action and gets a reward or penalty; the goal is to learn a policy that achieves the largest reward over time.
14. Batch: trains on all the data at once (offline) and is then frozen; with new data you must retrain from scratch. Online: learns gradually with every new data point/mini-batch.
15. How fast the system adapts to changing data. High: adapts quickly and forgets the old quickly (sensitive to noise). Low: slower but more stable.
16. Training on huge data that does not fit in memory: load a part, train on it, and repeat until the data is finished (also offline).
17. Instance-based memorizes the examples and compares by similarity (KNN). Model-based builds a model with parameters and predicts with it (Linear Regression).
18. The cost function = a function that measures the model's badness based on the parameters. Parameters: the model learns them. Hyperparameters: you set them before training.
19. GD adjusts the parameters gradually against the direction of the gradient until it reaches the minimum. η very small = very slow. η very large = jumps over the valley and may diverge.

20. An epoch = one full pass over the data. 500 ÷ 100 = 5 steps.
21. Data: insufficient quantity, nonrepresentative, poor quality, irrelevant features. Algorithm: overfitting and underfitting.
22. Overfitting: the model memorized the training set and does not generalize (solution: simpler/more data/clean/regularization). Underfitting: too simple (solution: stronger/better features/reduce the constraints).
23. Constraining the model to simplify it and reduce the risk of overfitting.
24. To measure performance on new data: training for training, validation for choosing the model/tuning hyperparameters, test once for the generalization error (the error rate on new cases).
25. Sampling bias: the sample does not represent the population. Data mismatch: the training data is different from the real-world data the model will work on.